VERA-PQ: A Post-Quantum, Hash-Only Smart Contract with Staking and Slashing for Verifiable Edge Aggregation in Blockchain-Based Federated Learning

Manojkumar Ashokkumar
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
[email]

***Abstract*—Federated learning (FL) over Internet-of-Things (IoT) devices depends on an edge aggregator that every client must trust, while the cryptography protecting such systems is being migrated to post-quantum (PQ) primitives that are costly on microcontrollers and on blockchains. Verifiable-FL designs that rely on zk-SNARKs add seconds to minutes of proving time and rest on pairing assumptions that a quantum adversary can break. We present VERA-PQ, a framework that spans blockchain, decentralized AI, IoT/edge computing and post-quantum cryptography. Each client commits to its quantized update with a Merkle tree and signs a single 32-byte digest per round with ML-DSA (FIPS 204). The gateway computes a coordinate-wise trimmed mean, commits two Merkle roots to a smart contract, and is then challenged on k coordinates derived from the hash of the commit block; the contract re-executes the trimmed mean on those coordinates using only Keccak-256 and integer arithmetic. The same contract holds an ETH bond from the gateway for every round: an accepted audit releases the bond and pays a reward shared with the devices, while a rejected or expired round slashes it, and we derive the bond that makes cheating unprofitable. In a Python/Solidity prototype with eight clients, the audit accepted all 210 honest rounds, rejected all 210 rounds in which the gateway silently replaced the trimmed mean with plain averaging, and detected sparse tampering at the rate predicted by $1-(1-f)^k$; over 100 rounds of 10% tampering the gateway lost 84 ETH of stake while earning 0.8 ETH. Building the audit response took at most 3 ms, whereas on-chain verification with reward settlement cost about 1.4 M gas for $n = 8$ and $k = 16$, roughly six times a Groth16 verification.**

***Index Terms*—federated learning, post-quantum cryptography, ML-DSA, blockchain, Ethereum, smart contracts, staking, slashing, cryptoeconomics, Internet of Things, robust aggregation, verifiable computation.**

I. Introduction

Four technology domains now meet in the same deployments: blockchains used as tamper-evident audit layers, decentralized and federated AI, IoT and edge computing, and the migration to quantum-resistant cryptography. NIST's draft transition plan deprecates RSA and elliptic-curve signatures after 2030 and disallows them after 2035 [1], the UK NCSC targets complete migration by 2035 [2], and recent resource estimates place RSA-2048 within reach of fewer than one million noisy qubits [3]. Systems built today on IoT fleets and immutable ledgers will still be running when these deadlines arrive.

In this paper we take the four domains to be (i) blockchain/Web3, (ii) decentralized and federated AI, (iii) IoT/edge computing, and (iv) quantum computing and post-quantum cryptography. Federated learning [4] is a natural fit for IoT intrusion detection because raw traffic never leaves the device, and blockchains have been proposed to coordinate and record such training [5]. Combining the four, however, exposes limitations that overlap across domains:

L1 — Untrusted aggregation. Robust aggregators protect the global model from poisoned clients only if the edge aggregator actually runs them. Clients and model consumers cannot observe this, and recording updates on a ledger does not show that the aggregate was computed correctly. In most designs a misbehaving aggregator also loses nothing, because it has no value at stake (AI × IoT × blockchain).

L2 — PQ cost on devices and ledgers. An ML-DSA-44 signature is 2,420 bytes against 64 bytes for ECDSA [6]; on a Cortex-M0+ class microcontroller ML-DSA signing averages about 159 ms with long rejection-sampling tails [7], and the smallest devices have tens of kilobytes of RAM [8]. On Ethereum, a Falcon-512 verification written in Solidity costs about 4.7 M gas versus 3,000 gas for ecrecover [9], and the ML-DSA precompile is still a draft [10] (quantum × IoT × blockchain).

L3 — Cost and quantum exposure of verifiability. Zero-knowledge machine-learning systems are limited mainly by proving cost [11]; VerifBFL, for example, needs up to 81 s per local-training proof and up to 2 s per aggregation proof [12]. Widely deployed SNARKs such as Groth16 (about 220 k gas to verify [13]) rely on pairings, which a quantum adversary breaks (AI × blockchain × quantum).

Our starting observation is that the robust aggregators used in practice, such as the coordinate-wise trimmed mean and median [14], are *order statistics evaluated independently per coordinate*. Output coordinate j depends only on coordinate j of the inputs. A verifier can therefore check a random subset of coordinates by re-executing the aggregator on values opened

from Merkle commitments [15], using nothing but a hash function. No prover, trusted setup or pairing is needed.

Building on this observation, this paper makes five contributions:

C1. VERA-PQ, a protocol that binds per-client ML-DSA signatures to Merkle commitments of quantized updates, derives an audit challenge from the ledger after the aggregator has committed, and accepts a federated round only if a smart contract's recomputation of the trimmed mean on the sampled coordinates matches (Section IV).

C2. A working prototype: a Python reference implementation and a Solidity contract that run on a simulated ledger and on an in-process Ethereum Virtual Machine (EVM), with liboqs and pure-Python ML-DSA back ends (Section VI).

C3. A security analysis giving the detection bound, the anti-grinding rules and an explicit list of non-goals (Section V).

C4. An evaluation of model accuracy under client poisoning and gateway misbehaviour, detection rate, computational overhead and gas cost, together with a trade-off comparison against SNARK-, STARK- and signature-only designs (Section VII). Limitations and deployment bottlenecks follow in Section VIII.

C5. A crypto-economic layer in the same contract: the gateway bonds ETH for every round, accepted rounds pay a reward split between the gateway and the devices, and failed or expired audits slash the bond with a bounty for the keeper who reports the expiry. We derive the minimum bond that makes cheating unprofitable and measure the outcome on the EVM (Sections IV-E, V-D and VII-F).

II. Background and Related Work

A. Blockchain-Based Federated Learning

Blockchains have been used to replace the central FL server, record model updates and reward participants. Biscotti [16] combines a ledger with Multi-Krum and verifier committees, and a 2023 survey of blockchained FL for IoT [5] lists overhead and compatibility as open problems. In most of these systems the ledger stores updates or models, while correctness of the aggregate is assumed or delegated to consensus. Bonding and slashing are standard tools for making misbehaviour costly: proof-of-stake Ethereum slashes validators that sign conflicting messages [17], and Truebit uses deposits and verification games to check outsourced computation [18]. VERA-PQ applies a bond to the aggregator itself and ties slashing to a hash-only audit, so the penalty is triggered automatically by the contract rather than by a dispute.

B. Robust Aggregation and Poisoning

Krum [19], the coordinate-wise trimmed mean and median [14], FLTrust [20] and FoolsGold [21] limit the influence of malicious clients. Local model-poisoning attacks can still degrade them [22], and a recent benchmark evaluates 15 attacks against 17 defenses [23]. These defenses all assume that the aggregator executes them faithfully; VERA-PQ removes that assumption for order-statistic aggregators.

C. Verifiable Aggregation

zkFL [24] has the aggregator produce a zero-knowledge proof of correct aggregation that blockchain miners verify. Xing *et al.* [25] and VerifBFL [12] also prove local training with zk-SNARKs, and RiseFL [26] reduces the cost of input checks. A survey of ZK-based verifiable ML [11] identifies proving cost and circuit expressiveness as the main barriers. TrustChain [27] audits aggregators statistically with an independence test rather than cryptographically. The closest design to ours is arya-STARK [28], which combines Dilithium signatures with STARK proofs of trimmed-mean aggregation; it gives deterministic soundness at the cost of full proof generation, whereas VERA-PQ uses no prover at all and accepts probabilistic soundness. Other recent designs verify robust aggregation with multi-key homomorphic encryption across several servers [29], with non-interactive zero-knowledge proofs over a cosine-similarity filter [30], or, in martFL, with a zero-knowledge proof of weighted aggregation for data marketplaces [31]. Nautilus [32] comes closest in spirit: it spot-checks a random 5% of updates against an on-chain commitment, but the checked property is that a node followed its training and compression policy, not that the aggregator computed an order statistic correctly, and the challenged node answers with a zk-SNARK.

D. Post-Quantum Cryptography on Ledgers and Devices

ML-DSA is standardized in FIPS 204 [6]. Surveys of post-quantum blockchains [33], [34] highlight the size and verification cost of PQ signatures, and hash-based multi-signatures have been proposed for Ethereum consensus [35]. A draft EIP prices an ML-DSA precompile at 4,500 gas [10]. On Cortex-M4, pqm4 measures ML-DSA-44 signing at about 3.9 M cycles [36], and ML-DSA-87 needs up to 119.6 KB of stack on a Cortex-M0+ [7]. In FL, BFL-MEC [37] and PQS-BFL [38] sign updates with Dilithium/ML-DSA but do not verify the aggregation, while ZKFL-PQ [39] proves norm bounds on updates with lattice-based ZK and has no ledger.

E. Sampling-Based Verification

Spot-checking with random challenges is classical in provable data possession [40]. A patent by Onai [41] verifies a sequential computation by sampling Merkle-committed checkpoints, and PoGO [42] uses Merkle proofs with random leaf checks to verify gradient training for consensus. VERA-PQ differs in what it samples: the output coordinates of a multi-input order statistic over PQ-authenticated client commitments, with the challenge seed taken from an external ledger block rather than from the prover's own commitment. Table I summarizes the coverage.

TABLE I
COVERAGE OF CLOSEST DESIGNS

Design	PQ auth.	Verif. agg.	Robust agg.	Ledger	Prover -free	IoT
Biscotti [16]	–	P	Y	Y	Y	–
zkFL [24]	–	Y	–	Y	–	–
VerifBFL [12]	–	Y	–	Y	–	P
TrustChain [27]	–	P	P	Y	Y	–
arya-STARK [28]	Y	Y	Y	–	–	–
BFL-MEC [37]	Y	–	P	Y	Y	P
PQS-BFL [38]	Y	–	–	Y	Y	–
ZKFL-PQ [39]	Y	P	P	–	–	–
VERA-PQ (ours)	Y	Y	Y	Y	Y	P

Y = provided; P = partial (e.g., statistical or committee-based checks, norm bounds only, or edge rather than device-class evaluation); – = not provided or not reported in the source. Entries reflect our reading of each paper's abstract and main text.

III. SYSTEM AND THREAT MODEL

A. Entities

The system has four kinds of entity (Fig. 1). **Clients** C_1, …, C_n are IoT or edge devices that train a local intrusion-detection model and hold an ML-DSA key pair; each device is registered on the ledger by the hash of its public key and a signature-algorithm identifier. The **edge gateway** G collects updates, aggregates them, posts commitments and answers challenges. The **ledger contract** L stores the registry and round commitments, derives challenges and runs the audit. **Model consumers** apply only aggregates whose round the contract has accepted.

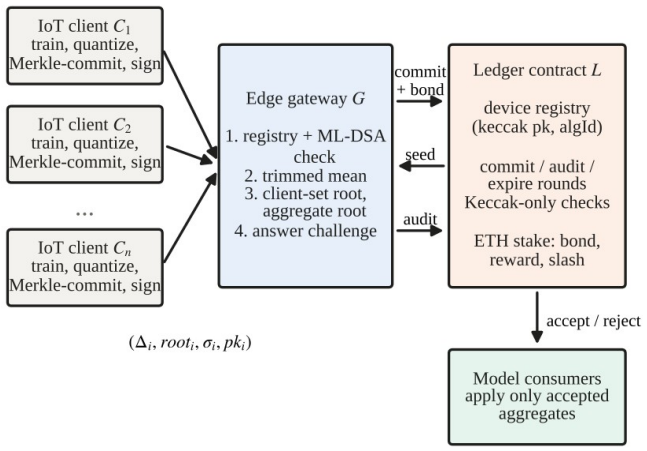


Fig. 1. VERA-PQ architecture. Clients send quantized updates with a Merkle root and one ML-DSA signature; the gateway aggregates and commits, the ledger derives a challenge seed from the commit block, and only audited rounds reach model consumers.

B. Adversary

A1 (Byzantine clients). Up to t of n clients send arbitrary updates. This is handled by the trimmed mean itself, which requires $n > 2t$, not by the audit.

A2 (malicious gateway). The gateway may skip robust aggregation and use a cheaper rule (the *lazy* gateway), alter any subset of output coordinates (*tampering*), substitute client values, drop or fabricate clients, or try to grind the challenge by re-committing.

A3 (quantum adversary). The adversary can break RSA, elliptic-curve signatures and pairing-based SNARKs, but not ML-DSA or Keccak-256 at their stated security levels.

C. Assumptions and Non-Goals

We assume collision resistance of Keccak-256, existential unforgeability of ML-DSA, correct execution of the ledger, and that the hash of the block containing a commitment cannot be predicted by the gateway when it commits. VERA-PQ does not hide update values (sampled coordinates are revealed on chain), does not prove that clients trained honestly, and does not make the host chain's own transaction signatures quantum-safe.

IV. THE VERA-PQ FRAMEWORK

A. Client Commitment and Signing

In round r, client i computes its update $\Delta_i \in \mathbb{R}^d$, quantizes it with scale $s = 10^6$ to integers $v_{i,j} = \text{round}(s\Delta_{i,j})$, and builds a Merkle tree whose leaves bind each value to its coordinate index:

$$\ell_{i,j} = H(v_{i,j} \,\|\, j), \quad \text{root}_i = \text{MerkleRoot}(\ell_{i,1}, \ldots, \ell_{i,d}) \tag{1}$$

where H is Keccak-256 and values are encoded as 256-bit two's-complement integers, matching the EVM. The client then signs one fixed-length digest:

$$m_i = H(\text{tag} \,\|\, r \,\|\, i \,\|\, \text{root}_i), \quad \sigma_i = \text{Sign}(sk_i, m_i) \tag{2}$$

where tag is the domain-separation string "VERA-PQ/update/v1" and Sign is ML-DSA-44. Because only the 32-byte digest is signed, the device performs one ML-DSA signature per round regardless of the model dimension d. The client sends $(\Delta_i, \text{root}_i, \sigma_i, pk_i)$ to the gateway.

B. Gateway Filtering and Aggregation

The gateway accepts an update only if $H(pk_i)$ matches the registry entry for i, σ_i verifies, and the Merkle root recomputed from the received values equals root_i. It sorts accepted clients by identifier and computes, for each coordinate,

$$a_j = \text{trunc}\left(\Sigma_{k=t+1}^{n-t} v_{(k),j} \,/\, (n - 2t) \right) \tag{3}$$

where $v_{(k),j}$ is the k-th smallest value at coordinate j and integer division truncates toward zero identically in Python and Solidity. The gateway builds a client-set tree with leaves $H(i \,\|\, \text{root}_i \,\|\, H(\sigma_i))$, so each signature is bound into the commitment, and an aggregate tree with leaves $H(a_j \,\|\, j)$. It then calls commitRound(r, R_C, R_A, n, d, t, k). The contract requires r to equal the next expected round and the previous round to be resolved.

C. Challenge and Audit

Once the commit block b_r is sealed, the challenge seed and sampled coordinates are

$$\text{seed}_r = H(\text{blockhash}(b_r) \,\|\, R_C \,\|\, R_A \,\|\, r) \tag{4}$$

and the sampled coordinates are $j_\tau = H(\text{seed}_r \,\|\, \tau) \bmod d$ for $\tau = 0, \ldots, k-1$. The gateway submits an audit transaction containing, for every client, its identifier, root_i, $H(\sigma_i)$ and a path into R_C; and, for every sampled coordinate, a_j with its path into R_A and all n client values $v_{i,j}$ with their paths into root_i. The contract runs Algorithm 1 (Fig. 2). Malformed openings revert the transaction, while a correctly opened but wrong aggregate marks the round rejected. If no audit arrives within 256 blocks

(the window in which blockhash is available), anyone can call expire(r), which also rejects the round.

Algorithm 1: Audit(r, client openings, samples) executed by contract L

Require: status(r) = committed; b_r < block.number ≤ b_r + 256
1: seed ← H(blockhash(b_r) ‖ R_C ‖ R_A ‖ r)
2: **for** c = 1 … n **do**
  3: require $id_c > id_{c-1}$ and pkHash[id_c] ≠ 0
  4: require MerkleVerify(R_C, $H(id_c \| root_c \| h_c)$, π_c)
5: **for** τ = 0 … k−1 **do**
  6: j ← H(seed ‖ τ) mod d
  7: require MerkleVerify(R_A, $H(a_j \| j)$, π_A)
  8: **for** c = 1 … n: require MerkleVerify($root_c$, $H(v_{c,j} \| j)$, $\pi_{c,j}$)
  9: **if** TrimMean$_t$($v_{1,j}$, …, $v_{n,j}$) ≠ a_j **then**
    10: status(r) ← rejected; emit RoundAudited(r, false, τ); **return**
11: status(r) ← accepted; emit RoundAudited(r, true, –)

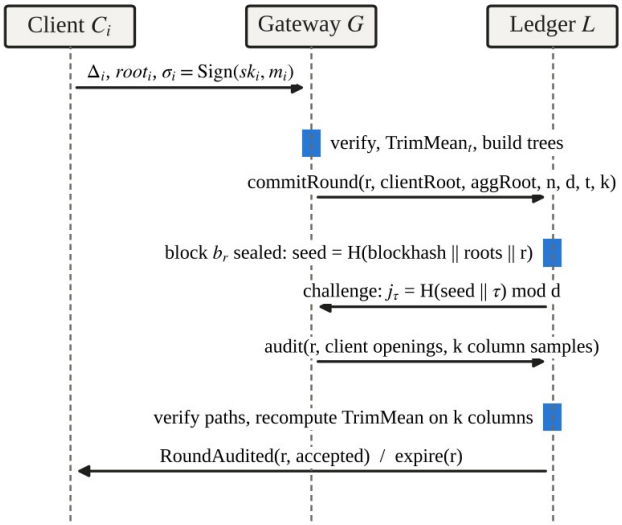


Fig. 2. Message flow of one VERA-PQ round. The challenge is fixed only after the commit block is sealed, so the gateway cannot know which coordinates will be opened when it commits.

D. Signature Spot-Check and Crypto-Agility

To catch fabricated clients, the seed also selects one client per round, which must reveal (pk, σ). The verifier checks that $H(pk)$ matches the registry, that $H(\sigma)$ matches the digest bound in the client-set leaf, and that σ verifies under ML-DSA. On a PQ-native chain or with the draft ML-DSA precompile [10] this runs on chain; in our prototype it runs in the simulated ledger. The registry stores a per-device algorithm identifier, so devices can migrate from classical to hybrid to ML-DSA-only authentication without redeploying the contract.

E. Crypto-Economic Layer: Staking, Rewards and Slashing

The contract is also an incentive mechanism denominated in ETH. A gateway first calls depositStake(); commitRound then requires at least B of unlocked stake and locks B (the bond) to round r. The operator funds a reward pool with fundRewards(), and each device registers a payout address. The round settles in the same transaction as the audit:

• **Accepted:** the bond is unlocked, and a reward R is taken from the pool, half credited to the gateway and half split equally across the n devices in the client set.
• **Rejected:** the bond is slashed from the gateway's stake into the reward pool, so a cheating gateway funds the rewards of future honest rounds.
• **Expired:** if no audit arrives within 256 blocks, anyone may call expire(r); the bond is slashed and 10% of it is credited to the caller as a keeper bounty.

All payments are credited to an internal balance and withdrawn later with claim(), a pull-payment pattern that keeps external calls out of the audit transaction and closes the usual re-entrancy path. Stake can be withdrawn only when it is not locked to an unresolved round. Our prototype uses B = 1 ETH and R = 0.1 ETH.

V. Security Analysis

A. Binding

Each sampled value is bound to the client's signed root by its Merkle path, each root is bound to σ_i by the signature and by $H(\sigma_i)$ in the client-set leaf, and each aggregate value is bound to R_A. Substituting any value at a sampled coordinate therefore fails path verification deterministically under collision resistance of H.

B. Detection Probability

Suppose the gateway reports wrong aggregates on m of the d coordinates, a fraction $f = m/d$. Because the challenge is unpredictable at commit time and indices are sampled uniformly with replacement, one round detects the deviation with probability

$$P_{\text{det}} = 1 - (1 - f)^k \tag{5}$$

so a target detection probability P requires

$$k \geq \ln(1 - P) / \ln(1 - f) \tag{6}$$

For P = 0.95 this gives k = 29 when f = 10% and k = 299 when f = 1%. A lazy gateway that replaces the trimmed mean with the plain mean produces a different integer on almost every coordinate whenever the extreme values are not symmetric, so it is caught with probability close to one even for small k.

C. Grinding and Withholding

A gateway might try to re-commit until it obtains a favourable challenge. Strictly sequential round identifiers, the requirement that the previous round be resolved, and the expiry rule together mean that each committed round is either audited against the challenge it received or rejected. A block proposer colluding with the gateway can still bias blockhash; production deployments should use RANDAO, a VRF or a commit-reveal beacon instead.

D. Economic Security

Let a cheating gateway gain G per round (for example, the computation it saves or a payment from an attacker), and let $P = P_{\text{det}}$ from (5). An honest round pays R. A cheating round pays $R + G$ if undetected and loses B if detected, so cheating is unprofitable when $(1 - P)(R + G) - P \cdot B \leq R$, that is, when

$$B \geq G(1 - P)/P - R \tag{7}$$

For a lazy gateway $P \approx 1$ and any positive bond suffices. For 10% tampering with k = 16 on d = 31, $P \approx 0.80$, so a bond of a quarter of the gain is enough; for a single tampered coordinate

($P \approx 0.41$) the bond must be about $1.5G$. Selective withholding does not help: a gateway that sees an unfavourable challenge and refuses to answer loses B through expire(), exactly as if it had been caught, and the keeper bounty means some party is always paid to trigger the expiry.

E. Post-Quantum View

The audit path uses only Keccak-256 and integer arithmetic. Generic quantum attacks reduce its preimage resistance to about 2^{128} (Grover) and its collision resistance to about 2^{85} (Brassard–Høyer–Tapp), and client authentication uses ML-DSA. The construction therefore relies on no pairing or discrete-logarithm assumption. The overall system is not fully post-quantum on Ethereum, however, because the host chain still signs transactions with secp256k1.

VI. Implementation

The prototype consists of a Python package (hashing, Merkle trees, PQ signatures, protocol, simulated ledger, EVM ledger and FL modules) and a Solidity contract, VeraPQLedger, compiled with solc 0.8.26 using the via-IR pipeline. ML-DSA is provided by liboqs 0.15.0 [43] through liboqs-python 0.16.0.1, with dilithium-py 1.4.0 (a pure-Python FIPS 204 implementation) as a fallback; Keccak-256 comes from pycryptodome. The EVM back end deploys the contract on an in-process chain (py-evm through eth-tester 0.14.0b1 and web3 8.0.0); deployment costs 1.65 M gas. The simulated ledger mirrors the contract logic, including the stake, reward and slashing accounting to the wei, and additionally performs the ML-DSA spot-check. Ten unit tests (honest acceptance, full tampering, forged signature, substituted value, values not matching the signed root, fabricated client, reward payout, slashing, expiry with keeper bounty, and commit without sufficient stake) pass on both signature back ends, and the Python verifier and the contract agreed on every configuration in Section VII-E.

The FL task is binary intrusion-style classification with logistic regression. Because our three-day build window did not allow preprocessing a full IoT intrusion dataset, the experiments use a synthetic tabular dataset generated with scikit-learn (30 features, 12 informative and 6 redundant, 60/40 class balance, 2% label noise), split into 400 samples per client and a fixed 2,000-sample test set. A loader for Edge-IIoTset [44] is included but was not used for the results reported here. Each client runs two epochs of mini-batch SGD per round. Table II lists the parameters. All measurements in Section VII were taken on a 2-vCPU Intel Xeon at 2.80 GHz with 7 GB of RAM running Linux and Python 3.13.

Independent reproduction. Before the crypto-economic layer was added (it does not change audit outcomes), we also ran the demonstration scenarios and the unit tests on a second, independent Windows laptop (hardware details to be added) that used the pure-Python ML-DSA fallback, with eight clients, two sign-flipping clients, trim $t = 2$ and $k = 16$. The outcomes were identical to those on the first machine. An honest gateway was accepted in all five rounds, with accuracy rising from 0.885 to 0.906. A lazy gateway without the audit was accepted in all five rounds while accuracy fell to 0.113–0.132. The same lazy gateway with the audit, and a gateway tampering with 10% of the coordinates, were rejected in all five rounds, and all six unit tests passed. Only the timings differed. Client commit-and-sign took 31–71 ms per client in pure Python, against about 1.2 ms with liboqs, and gateway verification and aggregation took 75–95 ms (one round 282 ms) against about 8 ms. The signature back end therefore dominates device-side cost in an unoptimized deployment.

TABLE II
Experimental Parameters

Parameter	Value
Clients n / trim t	8 / 2 (E1, E2); 4–32 (E3, E4)
Model / dimension d	Logistic regression; 31 (E1), 256 (E2), 32–1024 (E3, E4)
Local training	2 epochs, SGD, lr 0.2, batch 64
Rounds / seeds	10 rounds × 3 seeds (E1); 1 round × 100 seeds (E2)
Attacks	Sign-flip (×8), Gaussian ($\sigma = 8 \cdot \mathrm{std}(\Delta)$)
Gateway modes	honest, lazy (plain mean), tamper (shift fraction f)
Samples k	16 (E1); 4–32 (E2, E4)
Signature	ML-DSA-44 (liboqs 0.15.0)
Quantization	$s = 10^6$, int256 leaves

VII. Results and Discussion

A. Signature Primitives

Table III compares the PQ signature schemes on our x86 machine. ML-DSA-44 signs in 0.06 ms with liboqs, but its signature is 38 times larger than a 64-byte ECDSA signature. Falcon-512 has a smaller signature (654 bytes, variable length) but 7.5 ms key generation and floating-point arithmetic, and SPHINCS+-128f produces 17 KB signatures. The pure-Python fallback is roughly 570 times slower than liboqs for signing. The ECDSA baseline uses a different library (Python cryptography) and is shown only for scale. For microcontrollers we rely on published measurements [36], [7]. VERA-PQ amortizes these costs by signing one 32-byte digest per round, rather than per message or per parameter.

TABLE III
Signature Cost on x86 (mean of repeated runs)

Scheme	pk (B)	sig (B)	sign (ms)	verify (ms)
ML-DSA-44 (liboqs)	1,312	2,420	0.061	0.026
ML-DSA-65 (liboqs)	1,952	3,309	0.069	0.036
Falcon-512 (liboqs)	897	654	0.257	0.048
SPHINCS+-SHA2-128f	32	17,088	10.29	1.19
ML-DSA-44 (pure Python)	1,312	2,420	34.97	12.65
ECDSA secp256k1	64	64	0.416	0.405

B. Model Protection Under Poisoning and Gateway Misbehaviour

Table IV and Fig. 3 report final test accuracy after ten rounds (mean ± standard deviation over three seeds). With an honest gateway the audit changes nothing: all 210 audited honest rounds were accepted and accuracy was identical to the unaudited trimmed mean. Without the audit, a lazy gateway that silently substitutes plain averaging lets one sign-flipping client

cut accuracy from 0.912 to 0.277, and two such clients cut it to 0.176. Gaussian poisoning is milder because the noise partly averages out (0.845 with three attackers). With the audit enabled, every one of the 210 rounds produced by the lazy gateway was rejected, including the attack-free runs, because the plain mean differs from the agreed trimmed mean as soon as the extreme values are asymmetric. The model therefore stayed at its initial state (accuracy 0.408) instead of being corrupted. The audit provides safety, not liveness: an operator who sees repeated rejections must replace the gateway. In a separate run where the gateway shifted 10% of the coordinates (d = 31, k = 16), all five rounds were rejected.

With three sign-flipping clients and t = 2, the trimmed mean itself fails (0.173), and the audit, correctly, accepts these rounds because the gateway did exactly what was agreed. The audit verifies execution of the chosen aggregator, not whether t was chosen well; t must be set at least as large as the expected number of malicious clients.

TABLE IV
FINAL ACCURACY AFTER 10 ROUNDS (N = 8, T = 2)

Attack	FedAvg gateway, no audit	Trimmed mean (honest)	Lazy gateway + audit
None	0.912 ± 0.031	0.913 ± 0.030	rejected 30/30
Sign-flip 1/8	0.277 ± 0.025	0.911 ± 0.031	rejected 30/30
Sign-flip 2/8	0.176 ± 0.020	0.907 ± 0.036	rejected 30/30
Sign-flip 3/8	0.172 ± 0.010	0.173 ± 0.017	rejected 30/30
Gaussian 1/8	0.905 ± 0.031	0.912 ± 0.031	rejected 30/30
Gaussian 2/8	0.886 ± 0.028	0.912 ± 0.031	rejected 30/30
Gaussian 3/8	0.845 ± 0.030	0.911 ± 0.026	rejected 30/30

Rejected rounds are not applied, so the audited model stays at its initial accuracy (0.408 ± 0.007). The honest-gateway column is identical with and without the audit (0 of 210 rounds rejected).

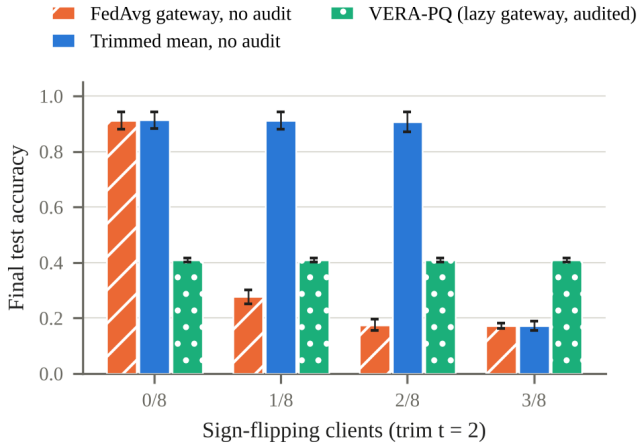


Fig. 3. Final accuracy versus number of sign-flipping clients. The audit cannot rescue a model once attackers exceed t, but it prevents a lazy gateway from silently applying a corrupted aggregate.

C. Detection of Sparse Tampering

Fig. 4 shows the empirical detection rate over 100 seeds for d = 256 against the analytic curve of (5). All sixteen configurations fall within 2.1 binomial standard deviations of the prediction; for example, f = 10% and k = 16 gives 0.82 measured against 0.820 predicted, and f = 25% and k = 16 gives 0.99 against 0.990. Very sparse tampering remains hard to catch: with f = 1%, even k = 32 detects only about a quarter of attempts in a single round.

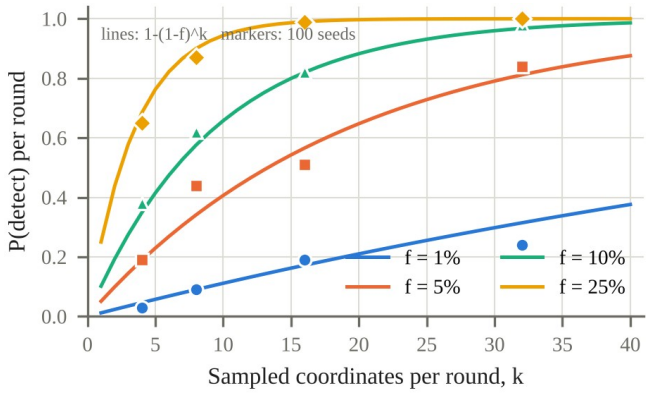


Fig. 4. Per-round detection probability versus number of sampled coordinates k (d = 256). Lines: $1-(1-f)^k$; markers: empirical rate over 100 seeds.

D. Computational Overhead

Table V breaks down the per-round cost. Client time covers building the Merkle tree and signing; it grows with d (0.8–1.2 ms at d = 32 to 16–18 ms at d = 1,024) because of hashing in Python, while the signature itself takes about 0.06 ms. Gateway verification and aggregation grows with $n \cdot d$ and reaches 610 ms for n = 32 and d = 1,024. Building the audit response, the analogue of proof generation, took at most 3 ms in every configuration, compared with up to 2 s for a SNARK aggregation proof in VerifBFL [12]; the hardware differs, so this comparison is indicative only. The audit payload grows from 41 KB to 232 KB of calldata across the grid.

TABLE V
PER-ROUND TIME (MS) AND AUDIT CALLDATA

n	d	Client	Gateway	Audit build	Calldata (KB)
8	32	1.21	8.2	0.64	41
8	1024	18.31	139.9	0.79	64
16	32	0.75	10.2	0.70	76
16	1024	16.20	328.6	1.67	120
32	32	0.86	21.4	1.17	147
32	1024	17.67	610.1	2.98	232

Client = Merkle commitment + ML-DSA signing; Gateway = signature and root checks + trimmed mean + trees. Python implementation, liboqs ML-DSA-44, mean of three rounds; calldata is the estimated ABI payload of the audit transaction.

E. On-Chain Cost

Committing a round costs about 149 k gas regardless of parameters, of which about 25 k is locking the bond. Fig. 5 shows that audit gas, including reward settlement, is almost exactly linear in $n \cdot k$ ($R^2 > 0.99$): about 10.5 k gas per client-sample plus 112 k for d = 32, and 16.1 k per client-sample plus 169 k for d = 1,024, the difference coming from longer Merkle paths. For the demonstration setting (n = 8, k = 16, d = 32) the audit costs 1.39 M gas, measured on the EVM, of which reward settlement adds about 4%, against roughly 220 k for a Groth16 verification with two public inputs [13] and 4.7 M for a single Falcon-512 verification in Solidity [9]. The largest configuration (n = 16, d = 1,024, k = 32) costs 8.46 M gas, which is impractical on a public layer-1 chain. Verifying eight ML-DSA signatures through the draft precompile would cost

about 36 k gas [10], but that would only authenticate the inputs, not check the aggregate.

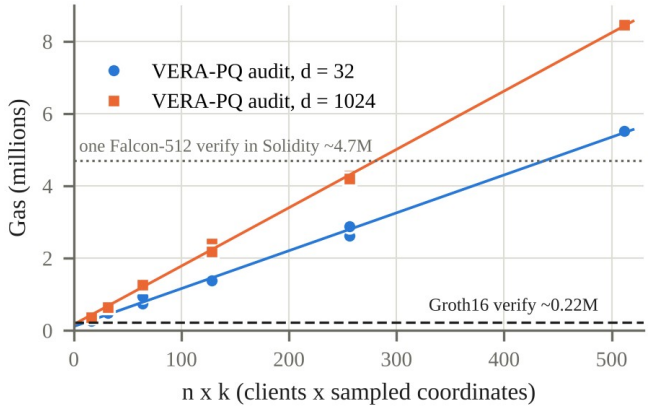


Fig. 5. Measured audit gas versus $n{\cdot}k$ on the EVM for $d = 32$ and $d = 1{,}024$, with linear fits. Horizontal lines give reference costs for a Groth16 verification and for one Falcon-512 verification in Solidity.

F. Crypto-Economic Outcomes

Table VI reports the gateway's balance sheet in each demonstration scenario (8 devices, 2 sign-flipping clients, $B = 1$ ETH, $R = 0.1$ ETH, 10 ETH initial stake). The no-audit baseline shows the core problem in economic terms: a lazy gateway poisons the model and is still paid the full reward. With the audit, the same gateway is slashed in every round. Over 100 rounds of 10% tampering ($k = 16$, $d = 31$) the gateway was caught and slashed in 84 rounds (80.4 predicted) and earned rewards from only 16, a net loss of 83.2 ETH against a 5 ETH profit for honest behaviour; with a single tampered coordinate it was slashed in 47 of 100 rounds (40.8 predicted). On the EVM, the stake, credit and pool balances matched the simulated ledger to the wei in every scenario we ran. The economic calls are cheap next to the audit: depositStake 45 k, fundRewards 44 k, expire with bounty 66 k and claim 29 k gas, while a rejected audit with slashing costs 1.25 M gas.

TABLE VI
GATEWAY BALANCE SHEET PER SCENARIO (SIMULATED LEDGER)

Scenario	Accepted	Slashed (ETH)	Gateway reward (ETH)	Final acc.
Honest, 5 rounds	5/5	0	0.25	0.906
Lazy, no audit, 5 rounds	5/5	0	0.25	0.132
Lazy + audit, 5 rounds	0/5	5	0	0.389
Withholding (expired), 3 rounds	0/3	3	0	0.389
Tamper $f = 10\%$, 100 rounds	16/100	84	0.80	–
Tamper 1 coord., 100 rounds	53/100	47	2.65	–

Devices receive the same total reward as the gateway in accepted rounds. In the withholding scenario the keeper earned 0.1 ETH per expired round. Final accuracy 0.389 is the initial model, because rejected rounds are never applied.

G. Trade-Off Comparison

Table VII places VERA-PQ among the alternatives. It trades the constant verification cost and deterministic soundness of SNARK-based verification for no prover time, no trusted setup and no pairing assumption. STARK-based designs such as arya-STARK [28] keep deterministic soundness and PQ plausibility but require proof generation. Signature-only PQ blockchain FL [37], [38] authenticates inputs cheaply but leaves aggregation unverified. Of the four, VERA-PQ is the only one whose security level can be tuned per round through k, which makes it best suited to permissioned chains or rollups where gas is cheap but proving hardware at the edge is not.

TABLE VII
QUALITATIVE TRADE-OFFS

Property	VERA-PQ	SNARK agg. proof	STARK proof	PQ-signed BFL
Soundness	$1-(1-f)^k$	Deterministic	Deterministic	None (agg.)
Prover cost	≤ 3 ms	Seconds	Seconds+	None
On-chain verify	0.3–8.5 M gas, $\propto n{\cdot}k$	≈ 220 k gas	Large proofs	Signatures only
PQ assumptions	Hash + ML-DSA	No (pairings)	Yes (hash)	Yes
Trusted setup	No	Groth16: yes	No	No
Update privacy	No	Yes	Possible	No

SNARK: zkFL, VerifBFL; STARK: arya-STARK; PQ-signed BFL: BFL-MEC, PQS-BFL. Values for other designs are taken from their papers and cited sources; only VERA-PQ was measured here.

VIII. LIMITATIONS AND DEPLOYMENT BOTTLENECKS

Gas grows linearly in $n{\cdot}k$. Beyond a few dozen clients the audit becomes too expensive for a public layer-1 chain. Deployments should use a permissioned chain or a rollup, commit chunks of c coordinates per leaf to reduce path overhead, or switch to an optimistic variant in which the audit runs only when a challenger disputes a round. None of these variants is implemented yet.

Soundness is probabilistic. Very sparse tampering ($f \approx 1/d$) evades a single audit with high probability. Repeated rounds, larger k and the slashable bond raise the cost of such attacks, but only in expectation.

Economic parameters. The bound in (7) needs an estimate of the gain G, which is hard to know in practice, and our B and R are illustrative. Rewards depend on an operator funding the pool, ETH price volatility changes the real value of the bond, and we did not simulate strategic agents or collusion between the gateway and devices. Clients themselves are not bonded; poisoning clients are handled by the trimmed mean, not by slashing.

No update privacy. Sampled update values are published on chain, which is incompatible with secure aggregation; order statistics cannot be computed over masked values. Differential privacy at the client is possible but reduces accuracy.

Randomness and host-chain security. A block proposer colluding with the gateway can bias blockhash. On Ethereum, transaction signatures remain secp256k1, the on-chain ML-DSA spot-check depends on a draft precompile [10], and the Falcon standard (FIPS 206) has not yet been published. Our EVM path therefore checks the binding of public keys only at the gateway.

Evaluation scope. The results use synthetic tabular data, a logistic-regression model with $d \le 1{,}024$, at most 32 clients and x86 hardware; no microcontroller was measured, so device costs are taken from the literature [36], [7]. Results on Edge-IIoTset [44], on deep models and on Raspberry Pi-class devices

remain to be produced. The trimmed mean is known to degrade under strongly non-IID data and when attackers exceed *t*. Finally, we deliberately left quantum machine learning out of scope: careful benchmarks find that classical baselines generally match or beat current QML models [45].

IX. Limitations and Contributions (Team Size: One)

The project was carried out by a single student within a three-day build window before the demonstration. The author is responsible for the protocol design, the Solidity contract with its staking and slashing layer, the EVM integration, the federated-learning pipeline and attack models, the experiments, the figures and this report. Working alone shaped the scope in three ways: we used synthetic data rather than a full IoT intrusion dataset, used an in-process EVM rather than a public testnet, and cited microcontroller measurements rather than collecting our own. Table VIII lists each area of work together with the limitation that follows from a one-person team.

TABLE VIII
Contributions and Team-Size Limitations

Work area	Limitation due to a one-person team
Protocol and security analysis	No independent review of the threat model or the detection bound; all arguments are by the author alone
Solidity contract and EVM path	Not audited or cross-implemented by a second developer; tested only on an in-process EVM
Staking and slashing layer	Parameters chosen illustratively; no game-theoretic simulation or external review of the incentive design
FL pipeline and attacks	One synthetic dataset and two attack types; no real IoT dataset yet
Experiments E1–E4	Measured on one machine and reproduced once on a second laptop; no third-party replication
Writing and figures	No internal peer review before submission

X. Conclusion and Future Work

VERA-PQ addresses three limitations shared by blockchain, federated AI, IoT and post-quantum cryptography: the untrusted aggregator, the cost of PQ signatures on devices and ledgers, and the cost and quantum exposure of zero-knowledge verification. Because robust aggregators are coordinate-wise order statistics, a smart contract can audit them from Merkle openings using only a hash function. Our prototype shows that this blocks a gateway that silently weakens the aggregator, detects tampering at the analytically predicted rate, and costs only milliseconds at the gateway. Because the same contract bonds the gateway's ETH and slashes it on a failed or expired audit, cheating also becomes unprofitable once the bond exceeds a bound set by the detection probability. The price is on-chain gas that grows with $n \cdot k$ and soundness that is probabilistic. Future work will add chunked leaves, an optimistic dispute mode, a game-theoretic study of the incentive parameters, evaluation on Edge-IIoTset and Raspberry Pi devices, and deployment on a chain with native ML-DSA verification.

References

[1] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, "Transition to post-quantum cryptography standards," NIST IR 8547 (Initial Public Draft), Nov. 2024. [Online]. Available: https://csrc.nist.gov/pubs/ir/8547/ipd

[2] UK National Cyber Security Centre, "Timelines for migration to post-quantum cryptography," 2025. [Online]. Available: https://www.ncsc.gov.uk/guidance/pqc-migration-timelines

[3] C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits," arXiv:2505.15917, 2025.

[4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. AISTATS, 2017, pp. 1273–1282.

[5] Y. Jiang, B. Ma, X. Wang, P. Yu, G. Yu, Z. Wang, W. Ni, and R. P. Liu, "Blockchained federated learning for Internet of Things: A comprehensive survey," arXiv:2305.04513, 2023.

[6] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," FIPS 204, Aug. 2024.

[7] R. Chhetri, "Benchmarking NIST-standardised ML-KEM and ML-DSA on ARM Cortex-M0+: Performance, memory, and energy on the RP2040," arXiv:2603.19340, 2026.

[8] C. Bormann, M. Ersue, and A. Keranen, "Terminology for constrained-node networks," IETF RFC 7228, May 2014.

[9] Ethereum Research forum, "The road to post-quantum Ethereum transaction is paved with account abstraction (AA)," thread started by user asanso, Feb. 2025; Falcon-512/1024 gas figures from a post dated Mar. 19, 2026. [Online]. Available: https://ethresear.ch/t/21783

[10] R. Dubois and S. Masson, "EIP-8051: Precompile for ML-DSA signature verification," Ethereum Improvement Proposal (Draft), Oct. 2025.

[11] Z. Peng, C. Zhao, T. Wang et al., "A survey of zero-knowledge proof based verifiable machine learning," Artificial Intelligence Review, vol. 59, art. 157, 2026; arXiv:2502.18535.

[12] M. A. Bouchiha et al., "VerifBFL: Leveraging zk-SNARKs for a verifiable blockchained federated learning," in Proc. IEEE/IFIP NOMS, 2025; arXiv:2501.04319 (the current arXiv version is titled "VerifBFL: A blockchain-based crowdsourcing framework for verifiable federated learning").

[13] Horizen Labs, "ZK analysis: The true cost of ZK verification," blog post, Dec. 3, 2025. [Online]. Available: https://www.horizenlabs.io/blog/zk-analysis-the-true-cost-of-zk-verification

[14] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in Proc. ICML, PMLR 80, 2018, pp. 5650–5659.

[15] R. C. Merkle, "A digital signature based on a conventional encryption function," in Advances in Cryptology—CRYPTO '87, LNCS 293, 1988, pp. 369–378.

[16] M. Shayan, C. Fung, C. J. M. Yoon, and I. Beschastnikh, "Biscotti: A blockchain system for private and secure federated learning," IEEE Trans. Parallel Distrib. Syst., vol. 32, no. 7, pp. 1513–1525, 2021.

[17] V. Buterin and V. Griffith, "Casper the friendly finality gadget," arXiv:1710.09437, 2017.

[18] J. Teutsch and C. Reitwießner, "A scalable verification solution for blockchains," arXiv:1908.04756, 2019.

[19] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in Proc. NeurIPS, 2017.

[20] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in Proc. NDSS, 2021.

[21] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "The limitations of federated learning in Sybil settings," in Proc. RAID, 2020, pp. 301–316.

[22] M. Fang, X. Cao, J. Jia, and N. Z. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in Proc. USENIX Security, 2020, pp. 1605–1622.

[23] H. Zhang, Y. Liu, X. He, J. Wu, T. Cong, and X. Huang, "SoK: Benchmarking poisoning attacks and defenses in federated learning," arXiv:2502.03801, 2025.

[24] Z. Wang, N. Dong, J. Sun, W. Knottenbelt, and Y. Guo, "zkFL: Zero-knowledge proof-based gradient aggregation for federated learning," IEEE Trans. Big Data (accepted); arXiv:2310.02554.

[25] Z. Xing, Z. Zhang, M. Li, J. Liu, L. Zhu, G. Russello, and M. R. Asghar, "Zero-knowledge proof-based practical federated learning on blockchain," arXiv:2304.05590, 2023.

[26] Y. Zhu, Y. Wu, Z. Luo, B. C. Ooi, and X. Xiao, "Secure and verifiable data collaboration with low-cost zero-knowledge proofs," Proc. VLDB Endow., vol. 17, p. 2321 ff., 2024; arXiv:2311.15310.

[27] E. Hallaji, R. Razavi-Far, and M. Saif, "TrustChain: A blockchain framework for auditing and verifying aggregators in decentralized federated learning," arXiv:2502.16406, 2025.

[28] A. A. Fall, "arya-STARK: Aggregation-robust yet authentic training via STARK proofs," IACR Cryptology ePrint Archive, Paper 2025/2238, 2025 (withdrawn May 2026).

[29] H. Kasyap, A. K. Pradhan, U. I. Atmaca, G. Cormode, and C. Maple, "PRoVeFL: Private robust and verifiable aggregation in federated learning," arXiv:2607.06612, 2026.

[30] "End-to-end verifiable and robust federated learning," arXiv:2609.15521, 2026 [authors to be added].

[31] "martFL: Federated learning for data marketplaces," arXiv:2309.01098, 2023 [authors and venue to be added].

[32] "Nautilus: A verifiable hierarchical federated learning framework for vehicular-edge-cloud systems," arXiv:2606.23017, 2026 [authors to be added].

[33] T. M. Fernández-Caramés and P. Fraga-Lamas, "Towards post-quantum blockchain: A review on blockchain cryptography resistant to quantum computing attacks," IEEE Access, vol. 8, pp. 21091–21116, 2020.

[34] Z. Yang, H. Alfauri, B. Farkiani, R. Jain, R. Di Pietro, and A. Erbad, "A survey and comparison of post-quantum and quantum blockchains," arXiv:2409.01358, 2024.

[35] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, "Hash-based multi-signatures for post-quantum Ethereum," IACR Communications in Cryptology, 2025; ePrint 2025/055.

[36] M. J. Kannwischer, M. Krausz, R. Petri, and S.-Y. Yang, "pqm4: Benchmarking NIST additional post-quantum signature schemes on microcontrollers," IACR ePrint 2024/112, 2024.

[37] R. Xu, S. R. Pokhrel, Q. Lan, and G. Li, "Post quantum secure blockchain-based federated learning for mobile edge computing," arXiv:2302.13258, 2023.

[38] D. Commey and G. V. Crosby, "PQS-BFL: A post-quantum secure blockchain-based federated learning framework," arXiv:2505.01866, 2025.

[39] E. Lansiaux, "Zero-knowledge federated learning with lattice-based hybrid encryption for quantum-resilient medical AI," arXiv:2603.03398, 2026.

[40] G. Ateniese, R. Burns, R. Curtmola, J. Herring, L. Kissner, Z. Peterson, and D. Song, "Provable data possession at untrusted stores," in Proc. ACM CCS, 2007, pp. 598–609.

[41] Onai Inc., "Verification of computation by untrusted source," U.S. Patent 10 873 462 B1, Dec. 2020.

[42] J. I. Orlicki, "PoGO: A scalable proof of useful work via quantized gradient descent and Merkle proofs," arXiv:2504.07540, 2025.

[43] D. Stebila and M. Mosca, "Post-quantum key exchange for the Internet and the Open Quantum Safe project," in Proc. SAC 2016, LNCS 10532, pp. 14–37, 2017.

[44] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," IEEE Access, vol. 10, pp. 40281–40306, 2022.

[45] J. Bowles, S. Ahmed, and M. Schuld, "Better than classical? The subtle art of benchmarking quantum machine learning models," arXiv:2403.07059, 2024.